Historical-Data Repair Contracts for Gemma Agents

A Calibrated Resource for Compatibility and Persistence Repair

Guilin Zhang

Abstract

We release eight executable historical-data repair contracts across six Python projects, covering 65 historical files and 252 dependent observations of reading, continued use and persistence. The resource calibrates acceptance when an old format has lost information. Two source repairs satisfy all 84 observations of a jsonpickle contract, yet literal equality with the reference rejects 60 observations for one valid repair. A corrupting control satisfies none despite completing every load and restart. All eight upstream-derived backports pass their contracts. Of 32 completed Gemma 4 repair attempts, four satisfy both semantic checks and source review. Released artifacts include historical bytes, patches, retained failures and an API-free verifier. The contribution is an executable acceptance contract and calibration, grounded in a known upstream ambiguity. Code, data and paper: https://github.com/GuilinDev/historical-data-repair-contracts

1. Introduction

Consider an old file containing a floating-point column. Two producer processes save the same values, one as float32 and the other as float64. Their complete files are identical (Figure 1). An evaluator that requires each original width asks the reader to distinguish inputs it cannot distinguish. Yet accepting any successful load is too weak: a reader can silently narrow a type or discard a later mutation.

Historical-data repair therefore needs an explicit acceptance contract. The test-oracle problem [1] is concrete here: which information survives in the file, which behavior must remain usable, and which alternative outputs are legitimate? We package those decisions with executable tasks rather than leaving them implicit in a single reference output.

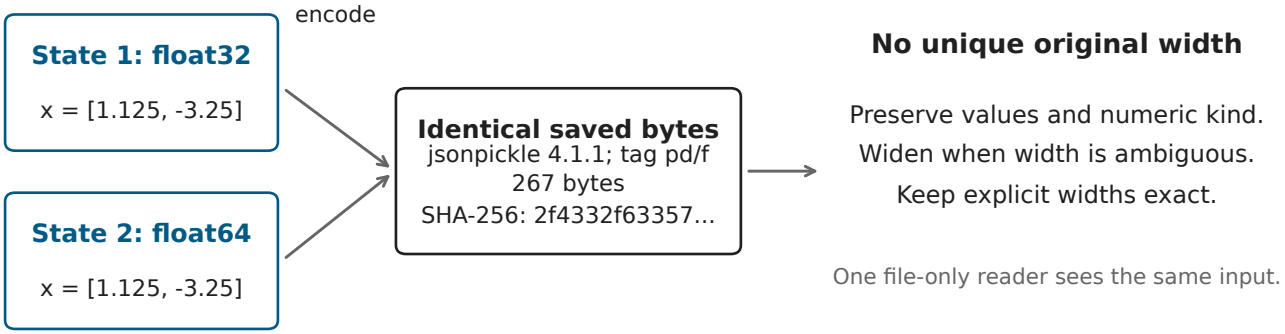


Figure 1. Two historical producer states yield the same complete document. Each state self-roundtrips with its original width, but the saved bytes cannot distinguish them. The pair was found in six targeted producer executions; this is an existence witness, not a collision-rate estimate.

Each task supplies an affected reader, historical producer source, public examples and a limited source edit scope. A defect lineage groups one upstream defect and its fix with the relevant producer and reader versions. Different files and process repetitions within a lineage extend coverage; they are not independent bugs. We contribute eight such task bundles, an acceptance calibration and a completed Gemma baseline. Table 1 combines resource coverage, control outcomes and model results.

2. Methods: executable repair contracts

The first four lineages in Table 1 were used during development; four were added prospectively, including two new projects. These public-source tasks are not uncontaminated held-out data. The 65 distinct files comprise 17 public and 48 withheld files, generated by genuine releases. H/F/R denote input regimes: historical files (131 observations), fresh API construction (53), and current-format roundtrips (68). Each includes declared use and fresh-process reload; they are not successive time steps.

Table 1. Resource, fresh-installation controls and all 32 completed attempts. Model cells count complete semantic and source-policy acceptances; control cells count observations.

	Resource				Controls: pass / N		Gemma: accepted	
Lineage	**Producer; base > fixed**	**Behavior exercised beyond loading**	**Files public / all**	**Checks H / F / R**	**Base**	**Fixed & backport**	**A**	**B**
Development-exposed lineages (4)								
packaging	25.0 / 26.0 26.1 > 26.2	Hash / comparison; collections	2/18	18/6/12	12/36	36/36 each	0/2	0/2
attrs	21.4.0 22.2.0 > 23.1.0	Slot state; cached hash	1/9	9/9/18	27/36	36/36 each	0/2	0/2
jsonpickle CSV	3.3.0 4.1.0 > 4.1.1	CSV text; mutated resave	2/6	6/6/6	12/18	18/18 each	0/2	1/2
SQLAlchemy	1.4.37 1.4.38 > 1.4.39	MutableDict; two commits	2/6	6/6/6	12/18	18/18 each	0/2	0/2
Newly added lineages (4)								
joblib endian	1.2.0 1.4.2 > 1.5.0	Writable mmap; flush	3/7	7/7/7	9/21	21/21 each	0/2	2/2
joblib compressed	1.2.0 1.4.2 > 1.5.0	Compressed load; raw mmap	3/7	7/7/7	9/21	21/21 each	1/2	0/2
jsonpickle dtype	4.1.1 4.1.1 > e7d95a6	Numeric kind / width	2/6	72/6/6	6/84	84/84 each	0/2	0/2
pandas	1.3.5 2.1.0 > 2.1.4	Frame operations; resave	2/6	6/6/6	12/18	18/18 each	0/2	0/2
Total	8 lineages; 6 projects		17/65	131/53/68	99/252	252/252 each	1/16	3/16

Base > fixed denotes affected reader > repaired upstream reader. Dtype H = 6 files x 12 processes; its base score varies by process. e7d95a6 is a pinned upstream commit. A: 26B A4B / PrimeIntellect BF16; B: 31B / CoreWeave FP4.

Contract example: a writable joblib mapping

The agent receives a joblib 1.2.0 file and may edit joblib/numpy_pickle.py (plus an optional test). In this public case, the affected reader changes 23 to 91234 in memory and in the explicit resave, but the work file still contains 23 after flush. Checking both files in new processes detects the lost write-through.

```
work = temporary_copy(historical_file)
a = joblib.load(work, mmap_mode="r+")
assert a.flat[1] == 23
a.flat[1] += 91211
a.flush()
joblib.dump(a, resaved_file)
for path in (work, resaved_file):
    b = fresh_process(joblib.load, path)
    assert b.flat[1] == 91234
```

Simplified from the executed verifier for one public case; fresh_process denotes a new interpreter. The full verifier also checks the remaining declared semantics.

3. Experiments: acceptance calibration

The jsonpickle defect was known upstream [2]: numeric tags can depend on runtime class-address order, which PYTHONHASHSEED does not fix. Our unmodified 4.1.1 producers generated the identical documents in Figure 1, while each self-roundtripped its original width. Let E_p be process p's encoder, and x_{32} and x_{64} be frames differing only in numeric width. Across processes p and q, $E_p(x_{32}) = E_q(x_{64})$. No file-only decoder can recover both original widths exactly.

For legacy numeric positions without explicit width suffixes, our contract requires exact values and numeric kind and permits nonnarrowing widths. Explicit historical widths and all new encodings remain exact. Differential testing already recognizes that different outputs can both satisfy a requirement [3]; the contribution here is an executable calibration of this particular contract.

We implemented three complete source-repair controls (Figure 2). The official conservative backport uses wide numeric types. Document exclusion v2 instead uses a producer invariant: each dtype name maps to one tag within a document. An explicit tag therefore excludes its dtype from another bare tag. A unique remaining candidate is used only for a nonempty, kind-compatible payload within the declared native-pandas numeric universe; otherwise the repair keeps the conservative fallback. A third repair deliberately narrows explicit float64 values.

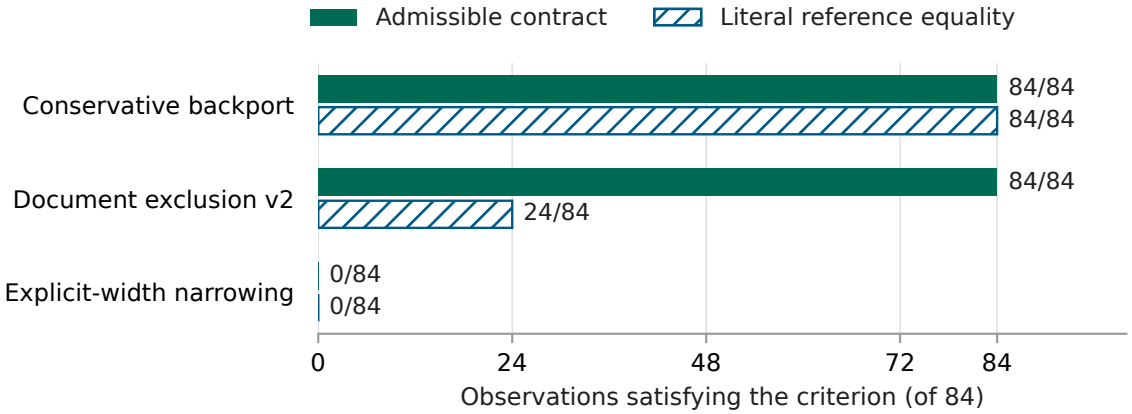


Figure 2. Complete source-repair controls on the numeric-code lineage. All three load and restart 84/84 observations. The alternate valid repair differs from literal reference output on 60 observations (five old files, twelve processes). These are researcher-built controls; literal equality is not the upstream test suite.

Both valid policies satisfy 84/84 observations. Literal equality with the official reader accepts only 24/84 for document exclusion. The remaining 60 are width-only differences on five files replayed in twelve processes; values, numeric kinds and subsequent operations agree with the contract. The narrowing control completes every load and restart but satisfies 0/84. Document exclusion uses explicit tags elsewhere in the same document. The witness in Figure 1 contains no such disambiguating tags, so this repair also falls back to float64.

These controls were written by the researchers, separately from Gemma. Source review found that v1 treated an empty column as evidence: Python's all(empty) is true, so a check intended to establish numeric type could pass without a value. We retained v1, guarded v2 against this case, and reran all 84 checks plus empty/nonempty pandas Interval columns, whose cells represent intervals. V2 matches the conservative reference but inherits its empty-Interval information loss. Arbitrary extension dtypes remain outside the validated scope.

4. Experiments: Gemma repair attempts

We ran Gemma 4 [4] on all eight tasks with two fresh conversations per deployment: A, 26B A4B through PrimeIntellect BF16; B, 31B through CoreWeave FP4. Provider and precision are confounded, hosted weight revisions are not attested, and public fixes may have appeared in pretraining.

Both deployments receive identical contracts, source tools and public checks, with up to 24 exploration rounds and 4,096 output tokens per exploration response. Both use temperature 0.2 during exploration and zero for the reserved 2,048-token final report. Exploration stops at a 120 KB request-context boundary. Acceptance requires every frozen observation and source review after delivery; the released protocol specifies tool, transport and retry limits.

All 32 conversations reached a valid final report and a decisive evaluation (Figure 3). Eight original PrimeIntellect rate-limit interruptions were resumed from identical saved requests, contexts and candidates, beyond the original three-retry ceiling. The failures and recovery journal remain in the release. Report completion records a finished attempt even when its delivered repair is incomplete.

	A: 26B A4B / BF16		**B: 31B / FP4**	
	Attempt 1	Attempt 2	Attempt 1	Attempt 2
packaging	12/36 U C	12/36 U C	12/36 U	24/36 E
attrs	27/36 U C	27/36 U C	27/36 U	27/36 E
jsonpickle CSV	12/18 U C	12/18 E	**18/18 PASS**	0/18 E C
SQLAlchemy	12/18 U	12/18 U	12/18 U	12/18 U
joblib endian	9/21 E	7/21 E	**21/21 PASS**	**21/21 PASS**
joblib compressed	**21/21 PASS C**	9/21 E	9/21 E C	9/21 E
jsonpickle dtype	0/84 U C	1/84 U C	0/84 U C	3/84 E C
pandas	12/18 U C	12/18 U C	12/18 E	12/18 E C

PASS: semantic + source acceptance; U: unchanged; E: edited, incomplete repair.
C: exploration context bound; it can accompany U, E or PASS.

Figure 3. Every completed Gemma conversation, with passed/declared observations from the original final evaluation. The dashed line separates development-exposed lineages (above) from newly added lineages (below). Counts are dependent coverage, not partial task successes. Providers and precision differ between A and B.

Four repairs were accepted: A solved compressed joblib loading once; B solved endian-map persistence twice and historical CSV dispatch once. The endian patches retain the file-backed array instead of returning a converted copy. The CSV patch dispatches the old CSV representation before JSON decoding. Residual redundant code and untested internal-call paths still limit production use. No attempt supplies an optional regression test. Main-run API cost is US$0.7785.

Trajectory diagnosis. Sixteen attempts leave source unchanged; twelve edit it without satisfying the full contract. The fifteen context-bound stops comprise ten unchanged, four edited-incomplete and one accepted attempt. Exact-text edits fail 78 times across eleven attempts, including only five of the sixteen unchanged attempts; these errors cannot explain all unchanged outcomes. Three attempts encounter UTF-8 search errors. In one SQLAlchemy conversation, twelve edits target a two-line snippet occurring twice and are rejected by the unique-match guard. The competition's edit_file interface likewise defaults to allow_multiple=False [5]. This motivates testing location-aware editing; our traces do not measure official-harness failure rates. An unchanged packaging attempt reads the same 100-line window eight times. After logged round 20, its next request reaches 125,578 bytes, above the 120,000-byte guard; round 21 is not sent. These are request bytes, not tokens or measured model capacity.

A separate CSV attempt passes all six historical initial checks but fails all six restarts and all six current-format cases. Its broad legacy-reader dispatch produces an index-length mismatch after resaving: initial semantics alone do not establish persistence.

5. Related work

Compatibility testing predates coding agents. DUPTester studies distributed-system upgrades [6]. OpenHands' persistence CI uses payloads from released PyPI versions, golden fixtures and release cutoffs [7]. SWE-bench [8] already includes historical ECSV compatibility, while GitChameleon studies version-conditioned API code generation [9]. Our scope is narrower: repairing a reader against immutable old artifacts and a declared persistence contract.

Repair overfitting [10] motivates evaluating beyond construction tests. SWE-Gate separates functional success from further constraints [11], and UTBoost strengthens SWE-bench evaluation through additional tests [12]. Mokav and ChangeGuard compare program behaviors [13,14]; metamorphic testing studies relations between executions [15]. Our save/reload checks and differential controls use established testing ideas.

The calibration tests both boundaries of acceptance: reject corruption and admit valid alternatives. A separate comparison with genuine upstream tests covers 12 development attempts from packaging and attrs (nine candidate snapshots). Both evaluators accept the same two and reject the same ten; no added rejection is observed among the two upstream accepts. A timed-out preservation grid is not counted as a semantic failure: a completed regression test already rejects that patch. These attempts are separate from the main 32, and Figure 2 compares literal outputs rather than authentic upstream suites.

6. Availability and limits

Resource card. Eight lineages, six projects, 65 files and 252 dependent observations. The release contains producer/reader source, reference and model patches, frozen expectations, licenses and retained failures. Original contributions use Apache-2.0; third-party licenses are retained. The qualified runtime is Linux ARM64 with Docker. Download and instructions: https://github.com/GuilinDev/historical-data-repair-contracts

Start with python3 compat.py integrity. This checks file hashes, not repair semantics. After the documented environment bootstrap, run python3 compat.py reproduce --task joblib_endian_1561 --controls base,fixed,reference for a complete task. A fresh dependency installation on the same ARM64 host reproduced all 24 control evaluations (756 observations). Release-only replay of 32 patches completed 1,008 observations and preserved the four full acceptances. Two incorrect dtype readers changed partial counts across processes; replay adds no model attempts.

Expected observations and admissibility were frozen before model calls; allowed-file upstream backports pass 252/252 checks. Historical bytes, producer source, expectations and verifier code remain immutable during repair. Agents edit designated Python files and may add a regression test. Execution uses network-disabled, read-only ARM64 containers with one CPU, 1 GB memory and temporary working copies. Source review excludes fixture hardcoding and verifier manipulation.

Limits. Eight selected lineages do not estimate defect prevalence or general coding ability. Most are upgrade regressions; compressed joblib loading is a longstanding defect, and dtype tags vary across processes. Two joblib fixes share a project. Only one lineage calibrates ambiguous outputs; its six files repeated across processes are not new artifacts. The edit allowlist supplies localization and excludes dependency/native-code fixes. Finite checks cover only declared behavior; other architectures remain unqualified.

7. Conclusion

• Repair contracts should test continued use and persistence, beyond successful loading.

• Acceptance calibration must reject corruption while admitting alternatives allowed by missing historical information.

• Eight executable tasks and 32 completed Gemma attempts provide a reproducible baseline: four repairs satisfy the full contract and source review.

Disclosure

Codex assisted with research, implementation, experiments, auditing and writing; Gemma generated the evaluated patches. The author is responsible for the claims and artifacts.

References

[1] Earl T. Barr et al. (2015). The Oracle Problem in Software Testing: A Survey. IEEE TSE 41(5):507-525. https://doi.org/10.1109/TSE.2014.2372785

[2] Theelx (2026). Fix bug with pandas dtypes decoding improperly. jsonpickle PR 634. https://github.com/jsonpickle/jsonpickle/pull/634

[3] William M. McKeeman (1998). Differential Testing for Software. Digital Technical Journal 10(1):100-107. https://www.cs.tufts.edu/comp/150FP/archive/bill-mckeeman/DifferentailTesting.pdf

[4] Gemma Team (2026). Gemma 4 Technical Report. arXiv:2607.02770. https://arxiv.org/abs/2607.02770

[5] Google and Kaggle (2026). Gemma 4 Developer Agent: Agent Tools. Competition documentation, accessed October 31. https://www.kaggle.com/competitions/gemma-4-developer-agent/overview/prizes

[6] Yongle Zhang et al. (2021). Understanding and Detecting Software Upgrade Failures in Distributed Systems. SOSP 2021. https://doi.org/10.1145/3477132.3483577

[7] OpenHands software-agent-sdk contributors (2026). test(ci): detect persisted settings compatibility regressions. OpenHands PR3298. https://github.com/OpenHands/software-agent-sdk/pull/3298

[8] Carlos E. Jimenez et al. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR 2024. https://arxiv.org/abs/2310.06770

[9] Diganta Misra et al. (2026). GitChameleon 2.0: Evaluating AI Code Generation Against Python Library Version Incompatibilities. ACL 2026. https://aclanthology.org/2026.acl-long.2170/

[10] Edward K. Smith et al. (2015). Is the Cure Worse Than the Disease? Overfitting in Automated Program Repair. ESEC/FSE. https://doi.org/10.1145/2786805.2786825

[11] Xin He et al. (2026). SWE-Gate: Passing Functional Tests Is Not Enough for Software Engineering Agents. arXiv:2609.04167. https://arxiv.org/abs/2609.04167

[12] Boxi Yu et al. (2025). UTBoost: Rigorous Evaluation of Coding Agents on SWE-Bench. ACL 2025. https://aclanthology.org/2025.acl-long.189/

[13] Khashayar Etemadi et al. (2025). Mokav: Execution-driven differential testing with LLMs. JSS 230:112571. https://doi.org/10.1016/j.jss.2025.112571

[14] Lars Gröninger et al. (2025). ChangeGuard: Validating Code Changes via Pairwise Learning-Guided Execution. PACMSE/FSE 2025. https://doi.org/10.1145/3715760

[15] Sergio Segura et al. (2016). A Survey on Metamorphic Testing. IEEE TSE 42(9):805-824. https://doi.org/10.1109/TSE.2016.2532875

Software acknowledgment

Figure preparation used Scientific Agent Skills: A Library of Procedural Knowledge for Research Agents (Kassis et al., 2026). https://doi.org/10.48550/arXiv.2609.00065